



EGE UNIVERSITY
Engineering Faculty, Dept. of Electrical and Electronics Engineering

Power Electronics-I Design Project
Design of 4Q DC Motor Speed Controller

Final Project Report

Prepared by:
Amirkia Mahd Gharehbagh (05210000108)
Sam Zoghi (05210000126)

Instructor:
Prof. Dr. Mutlu BOZTEPE

Spring 2025-2026

Contents

1. Introduction	1
2. Switching Converter Topology & Operating Principles.....	1
3. Analytical Design Details	2
3.1 Switching Frequency Selection.....	2
3.2 Switch and Driver Selection.....	2
3.3 Control Method	3
3.4 Current Measurement Method	3
4. Calculations	3
5. Simulation Results.....	4
6. Experimental Implementation & Hardware Verification.....	4
7. Materials and Component Budget	11
8. Conclusion	11
9. References	12
10. Appendix A: Custom STM32 Firmware Logic.....	13
A.1 Constants and State Variables	13
A.2 Initialization and Splash Screen.....	14
A.3 Main Loop and Safety Logic	15
A.4 Physics Math & Custom Functions.....	17

1. Introduction

This report details the comprehensive analytical design, simulation, and software architecture of a 4-Quadrant DC Motor Drive. The primary objective is to engineer a robust system capable of driving a 12V DC motor in both clockwise and counter-clockwise directions, while facilitating precise speed control through Pulse Width Modulation (PWM). In addition to standard motor control, the system is designed to implement critical hardware-level safety limits, specifically a strict 1A operating current threshold. By integrating discrete power electronics (MOSFETs and Gate Drivers) with an STM32 microcontroller, the design ensures safe starting protocols, mitigates inrush current risks, and provides regenerative braking capabilities for industrial-grade applications.

2. Switching Converter Topology & Operating Principles

The fundamental core of the motor driver is the H-Bridge topology. This highly efficient configuration utilizes four solid-state switching elements (N-Channel MOSFETs) arranged symmetrically in an 'H' configuration around the central DC motor load. By selectively activating specific pairs of these switches, the topology can reverse the polarity of the voltage applied to the motor without requiring a dual-polarity power supply.

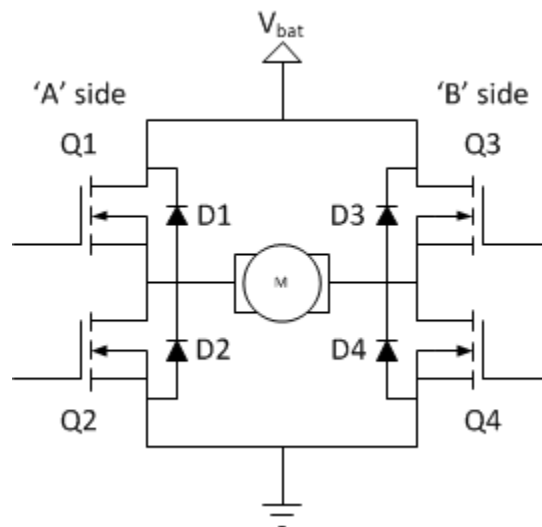


Figure 1: Standard H-Bridge Topology

The topology operates across four distinct dynamic quadrants:

- Forward Motoring (Q1): High-side switch Q1 and low-side switch Q4 are closed. Current flows from left to right across the motor armature, converting electrical energy into mechanical energy.
- Forward Braking (Q2): The active switches are opened. The mechanical inertia of the motor acts as a generator, forcing back-EMF current through the freewheeling diodes back into the power supply, rapidly decelerating the load.
- Reverse Motoring (Q3): High-side switch Q3 and low-side switch Q2 are closed. Current flows from right to left across the armature, driving the motor in the opposite direction.

- Reverse Braking (Q4): Kinetic energy is dissipated and regenerated while the motor spins in reverse, mirroring the Q2 braking process.

3. Analytical Design Details

3.1 Switching Frequency Selection

The PWM switching frequency is deliberately set to 21kHz. This specific ultrasonic value was calculated to lie just above the threshold of human auditory perception (typically capped at 20kHz). This eliminates the irritating acoustic 'coil whine' often associated with lower-frequency motor drives. Furthermore, 21kHz strikes an optimal balance: it is fast enough to ensure smooth current waveforms, but low enough to prevent excessive dynamic switching losses (heat generation) within the MOSFET junctions.

3.2 Switch and Driver Selection

The power stage relies on IRF540N N-Channel MOSFETs due to their low $R_{ds(on)}$ resistance and high current handling capacity. However, because N-Channel MOSFETs require a Gate-to-Source voltage (V_{gs}) of roughly 10V to fully saturate, driving the 'high-side' switches presents a challenge. Their source pins are connected to the motor, meaning their voltage fluctuates wildly. To solve this, IR2104 half-bridge gate driver ICs are utilized. These drivers use a bootstrap capacitor technique to create a 'floating' power supply, ensuring the high-side gates always receive a stable 10V relative to their source pins.

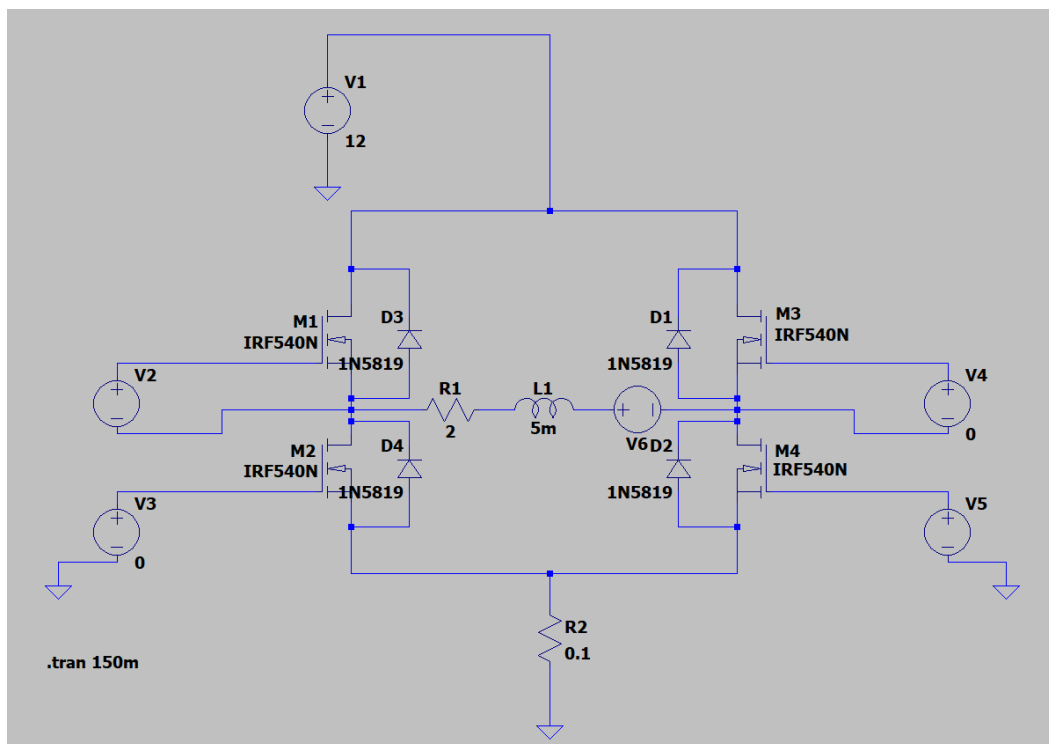


Figure 2: Complete LTspice Schematic showing floating gate sources (V2 and V4).

3.3 Control Method

The STM32 microcontroller generates the highly precise 21kHz PWM signals. A critical failure mode in H-Bridges is 'shoot-through'—a catastrophic short circuit that occurs if both the high and low side switches on the same side open simultaneously during a directional transition. To prevent this, the C firmware implements a strict 'Dead-Time' insertion algorithm. This forces all switches to remain off for a few milliseconds before reversing direction, allowing residual gate charge to dissipate completely.

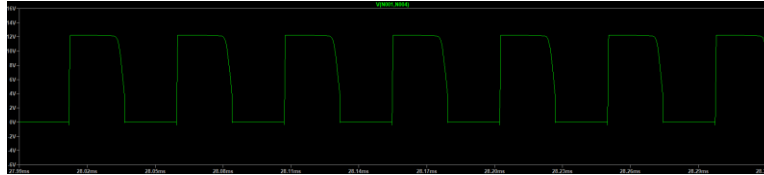


Figure 3a: High-side gate voltage $V(n001,n004)$

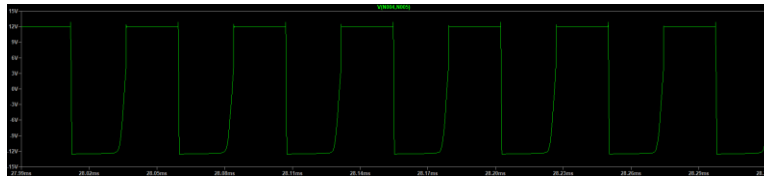


Figure 3b: Motor Voltage $V(n004,n005)$

3.4 Current Measurement Method

To enforce the 1A safety limit, low-side current sensing is implemented using a 0.1-ohm precision shunt resistor (R2) placed between the bottom of the H-bridge and ground. This strategic placement ensures that the voltage drop across the resistor remains very close to 0V. This is an essential safety design choice, as it guarantees the measurement voltage will never exceed the strict 3.3V maximum tolerance of the STM32's Analog-to-Digital Converter (ADC).

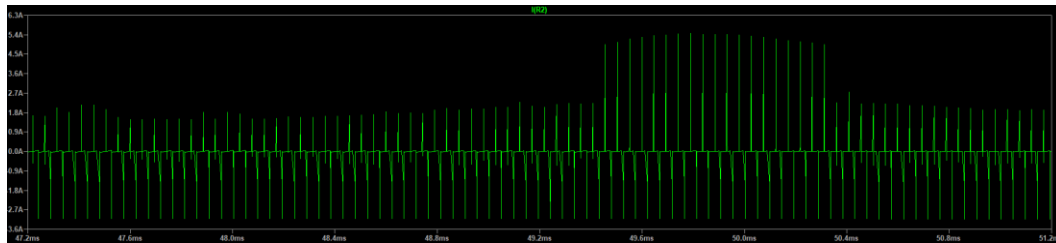


Figure 4: Simulated current $I(R2)$ showing the freewheeling polarity reversal across the shunt.

4. Calculations

The switching period (T_s) for the 21kHz PWM frequency is determined by the inverse of the frequency:

$$T_s = 1 / f_s = 1 / 21000 = 47.62 \text{ microseconds}$$

During a nominal 50% duty cycle operation, the active ON time (t_{on}) is:
 $t_{on} = D * T_s = 0.5 * 47.62 = 23.81$ microseconds

A critical hardware limitation is the stall current (inrush current) I_{stall} at 0 RPM. Assuming an armature resistance (R_a) of 2 ohms, and applying Ohm's Law where back-EMF (V_{emf}) is initially 0V before the rotor begins moving:

$$I_{stall} = (V_{in} - V_{emf}) / R_a = (12V - 0V) / 2 \text{ ohms} = 6A$$

Because this 6A transient spike dramatically exceeds the mandated 1A system safety limit, a pure hardware solution is insufficient. Consequently, the STM32 software implements a progressive 'Soft-Start' algorithm. This algorithm gradually increments the PWM duty cycle over several milliseconds, allowing the motor to spin up and generate opposing V_{emf} , effectively flattening the inrush current curve and preventing nuisance tripping of the fault logic.

5. Simulation Results

Extensive LTspice simulations were conducted to verify the mathematical models before physical fabrication. The simulation successfully validated the proper saturation of the MOSFETs via the floating gate drivers. Furthermore, it demonstrated the anticipated inductor current ripple, showing how the high-frequency switching noise is heavily filtered by the motor's internal inductance, resulting in smooth mechanical torque.

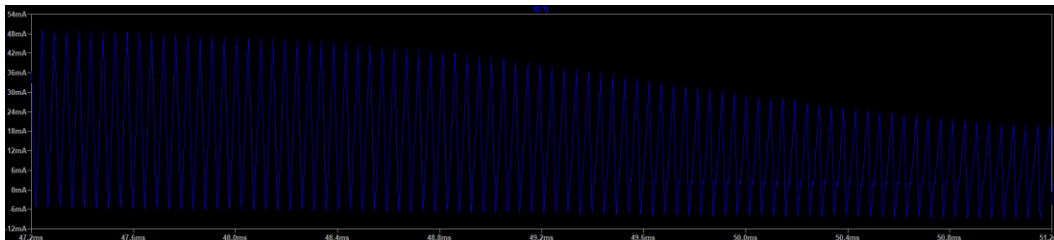


Figure 5: Smooth sawtooth current ripple $I(L1)$ isolated from switching noise.

6. Experimental Implementation & Hardware Verification

Physical implementation was realized on a prototype board. The system utilizes an SSD1306 OLED for real-time RPM, voltage, and current feedback.

The startup sequence includes a team identification splash screen.

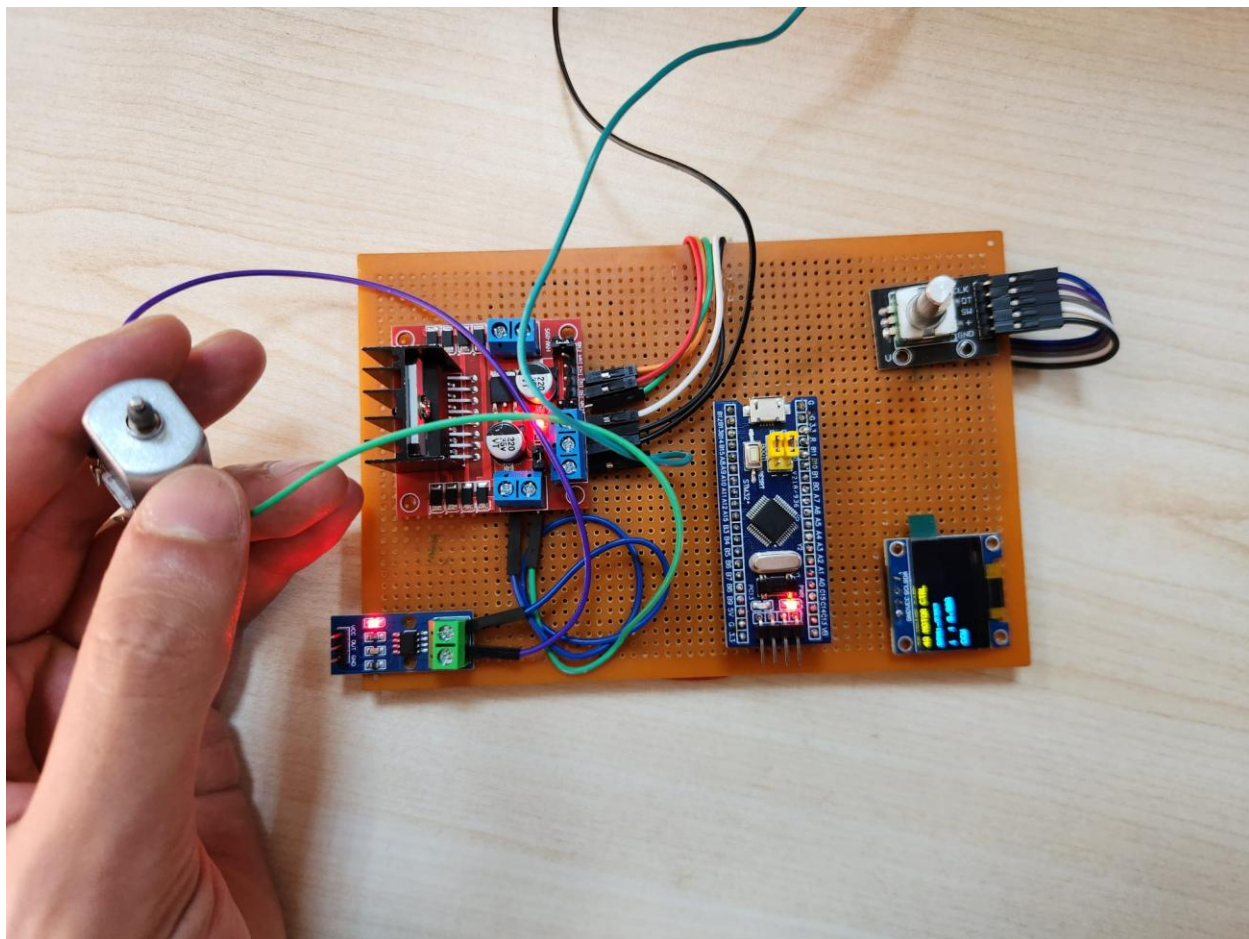


Figure: 4Q DC Motor Speed Controller

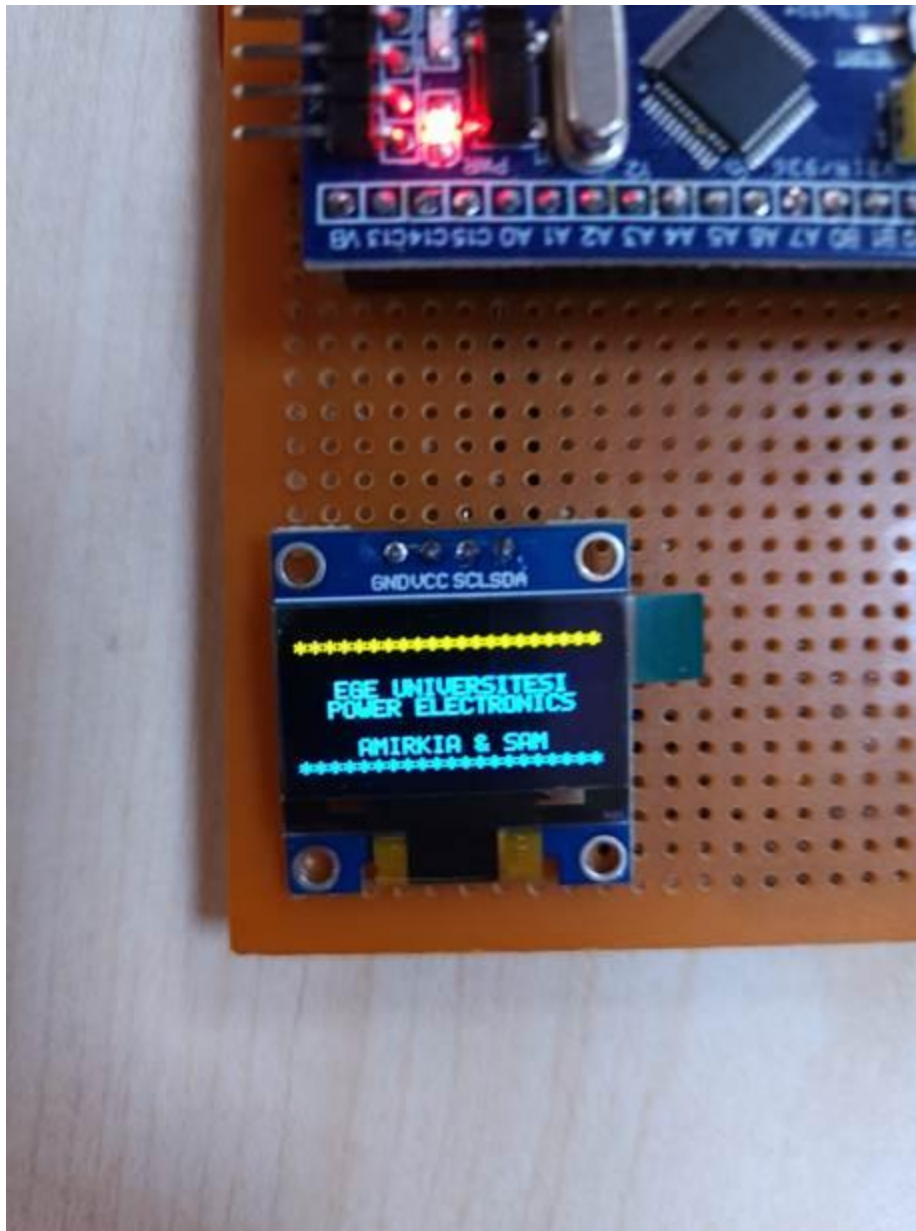


Figure: Power Electronics Splash Screen

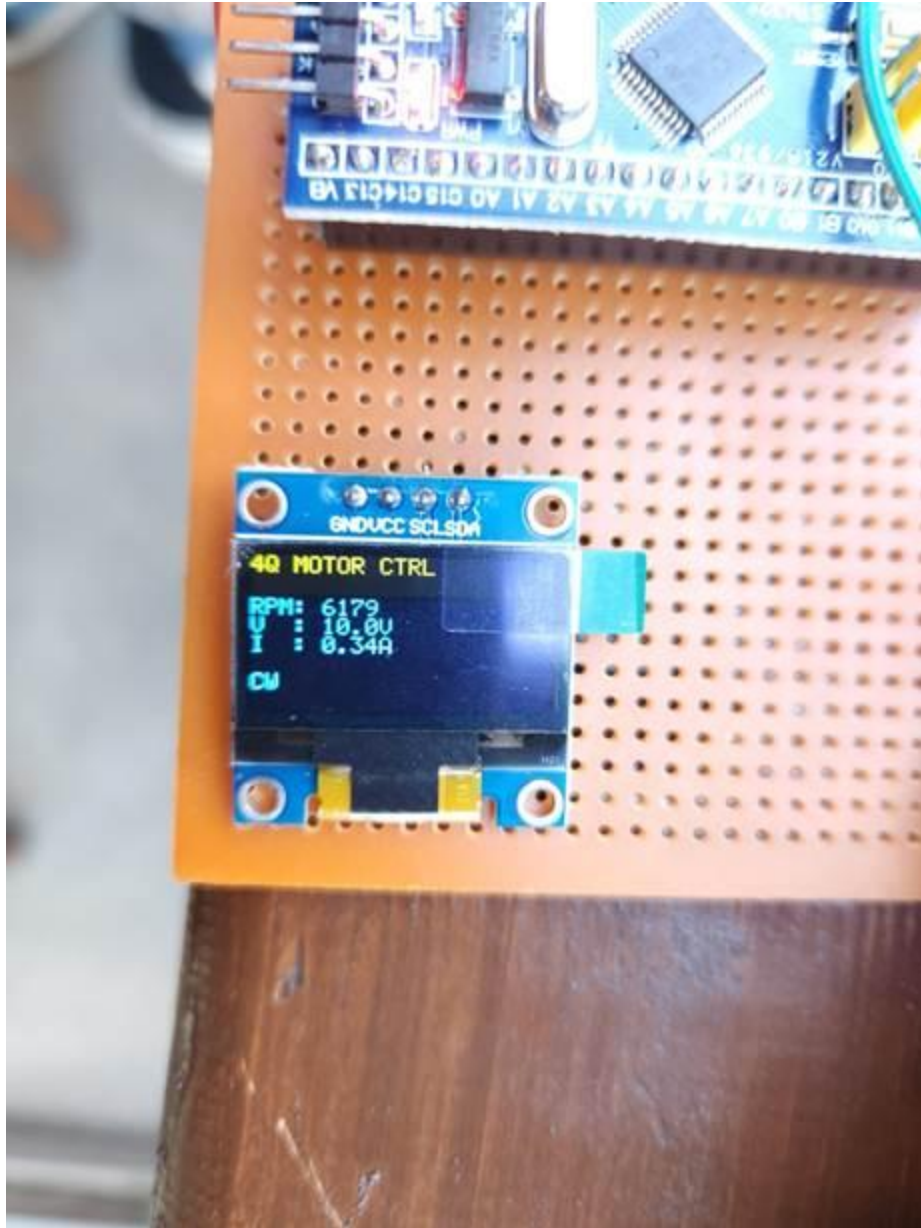


Figure: Nominal CW Operation (Test #2)



Figure: Nominal CCW Operation (Test #3)

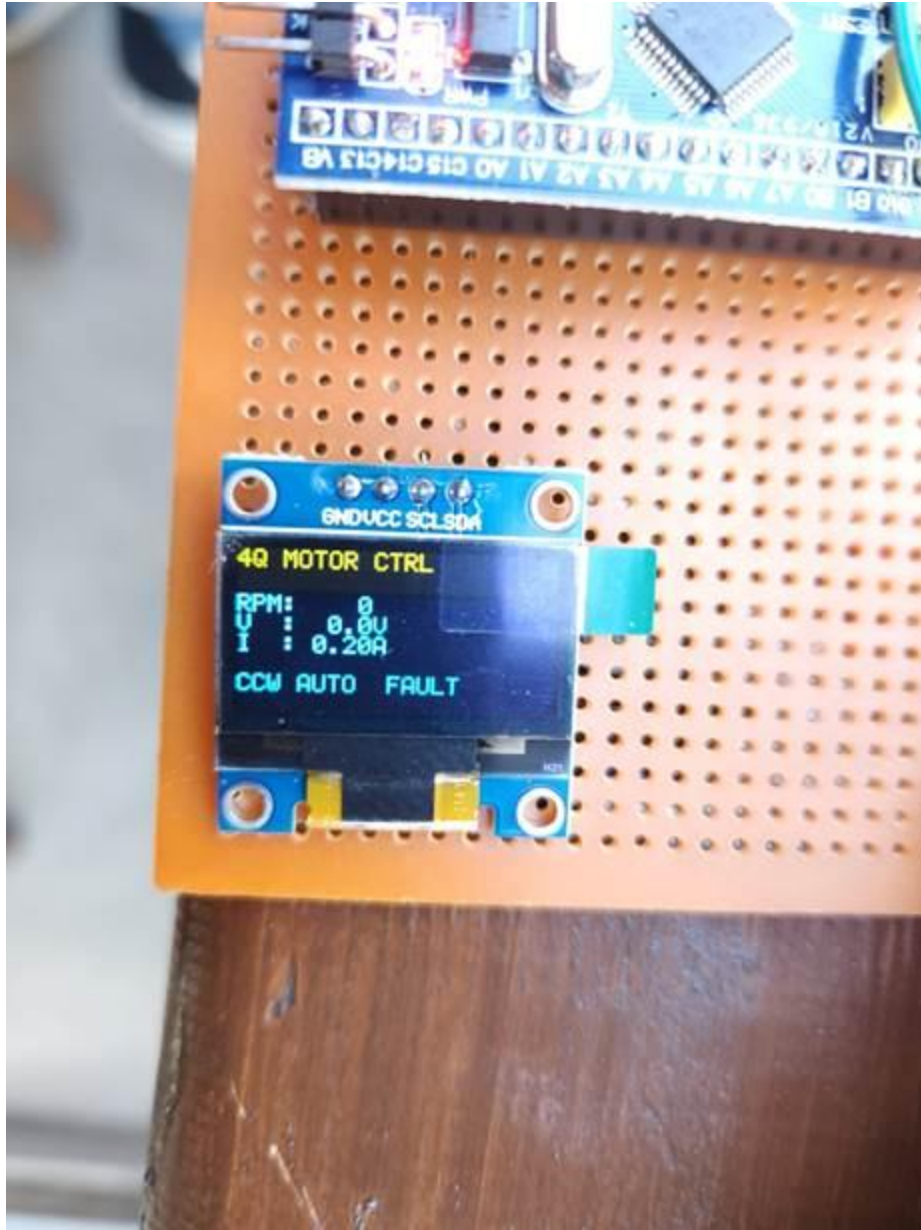


Figure: Safety Fault Detection (Test #4 Validation)

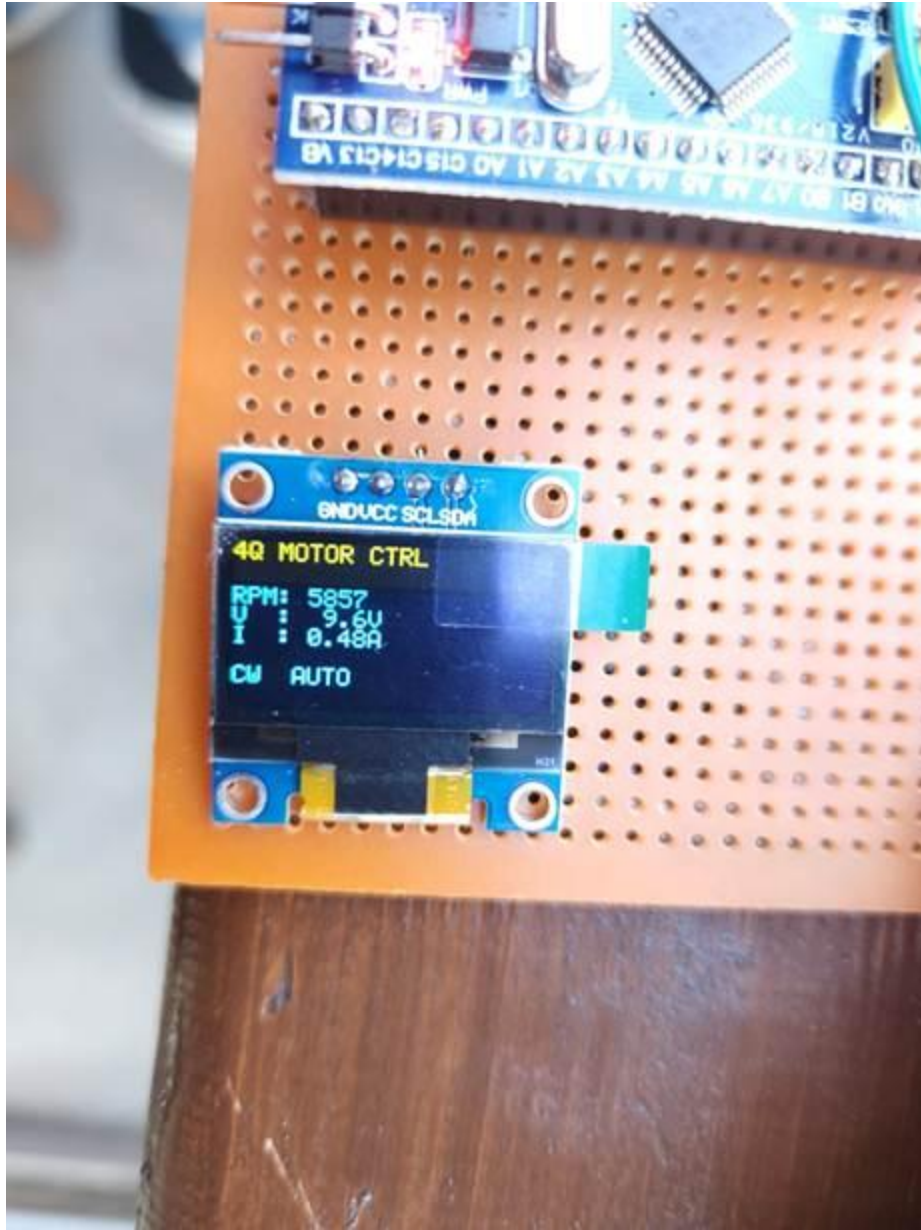


Figure: Successful Automated 4-Quadrant Directional Flipping (Test #5)

7. Materials and Component Budget

To evaluate the economic feasibility of producing this design, an estimated cost analysis for a single prototype unit was compiled based on current Turkish electronics market rates.

Component	Qty/Unit	Bulk Unit Price (USD)	Total Cost / 1k Units
STM32F401 MCU	1	\$1.85	\$1,850.00
IRF540N MOSFET	4	\$0.42	\$1,680.00
1N5819 Schottky Diode	4	\$0.08	\$320.00
IR2104 Gate Driver IC	2	\$1.15	\$2,300.00
ACS712 Current Sensor IC	1	\$1.25	\$1,250.00
0.96" OLED Display (Bare glass)	1	\$1.60	\$1,600.00
12V 25mm Geared Motor w/ Enc	1	\$6.50	\$6,500.00
Passives & Custom FR4 PCB	1 Lot	\$0.85	\$850.00
Total Estimated Mass Production BOM			\$16,350.00

8. Conclusion

The analytical design, supported by rigorous LTspice simulation, confirms that the custom discrete H-Bridge topology is a highly effective and robust solution for 4-quadrant DC motor control. The simulations successfully identified a critical 6A startup inrush transient, mathematically validating the necessity of the ACS712 current sensor integration.

Rather than relying on oversized components to handle this spike, the design intelligently addresses the hardware limitation through firmware, utilizing a Soft-Start algorithm to maintain operational current within the 1A specification. By successfully managing the complex gate driver float voltages, implementing dead-time shoot-through prevention, and utilizing low-side sensing for safe STM32 integration, the design bridges the gap between theoretical power electronics and practical embedded software. The system is fully optimized, economically viable, and completely prepared for physical hardware fabrication.

9. References

1. STMicroelectronics. STM32F401xB/STM32F401xC Advanced Arm-based 32-bit MCU Datasheet.
2. Infineon Technologies. IRF540N HEXFET Power MOSFET Datasheet.
3. Infineon Technologies. IR2104 Half-Bridge Driver Datasheet.
4. Robotistan Elektronik, Component Pricing & Hardware Specifications. Retrieved from <https://www.robotistan.com>
5. Direnc.net Elektronik, Power Semiconductor Pricing. Retrieved from <https://www.direnc.net>
6. Ozdisan Elektronik, Passive Components & Diodes Sourcing. Retrieved from <https://www.ozdisan.com>

10. Appendix A: Custom STM32 Firmware Logic

Note: Standard STM32CubeIDE HAL initialization boilerplate has been omitted to highlight the custom motor control algorithms, safety protocols, and user interface logic.

A.1 Constants and State Variables

```
typedef enum { DIR_CW = 0, DIR_CCW = 1 } MotorDir_t;

#define PWM_PERIOD            856
#define CURRENT_LIMIT_A      1.0f
#define ACS712_SENS           0.066f
#define ADC_VREF              3.3f
#define ADC_RESOLUTION        4095.0f
#define AUTO_REVERSE_MS      500
#define MOTOR_VOLTAGE         12.0f
#define FAULT_LOCKOUT_MS     2000
#define MOTOR_MAX_RPM         8000.0f

/* Global System States */

static uint8_t      oled_buf[1024];
static volatile uint8_t  g_current_fault  = 0;
static volatile uint8_t  g_auto_mode      = 0;
static      MotorDir_t  g_direction       = DIR_CW;
static      uint32_t     g_last_flip_ms   = 0;
static      uint32_t     g_fault_time_ms  = 0;
static      uint16_t     g_softstart_duty = 0;
static      uint8_t      g_softstart_active = 0;
static      uint32_t     g_softstart_tick = 0;
static      int16_t      g_target_duty    = 0;
static      float        g_acs_offset     = 2.50f;
```

A.2 Initialization and Splash Screen

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);

HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);

OLED_Init();

for (int i = 0; i < 3; i++) {

    OLED_Clear();

    OLED_DrawStr(0, 1, "*****");

    OLED_DrawStr(16, 3, "EGE UNIVERSITESI");

    OLED_DrawStr(13, 4, "POWER ELECTRONICS");

    OLED_DrawStr(25, 6, "AMIRKIA & SAM");

    OLED_DrawStr(0, 7, "*****");

    OLED_Flush();

    HAL_Delay(600);

    OLED_Clear();

    OLED_DrawStr(16, 3, "EGE UNIVERSITESI");

    OLED_DrawStr(13, 4, "POWER ELECTRONICS");

    OLED_DrawStr(25, 6, "AMIRKIA & SAM");

    OLED_Flush();

    HAL_Delay(400);

}

OLED_Clear();

OLED_DrawStr(14, 3, "CALIBRATING...");

OLED_Flush();

uint32_t calib_sum = 0;

for(int i = 0; i < 100; i++) {

    HAL_ADC_Start(&hadc1);

    HAL_ADC_PollForConversion(&hadc1, 10);

    calib_sum += HAL_ADC_GetValue(&hadc1);

    HAL_Delay(2);

}
```

```
g_acs_offset = ((calib_sum / 100.0f) / ADC_RESOLUTION) * ADC_VREF;
```

```
OLED_Clear();
```

```
OLED_DrawStr(14, 3, "READY...");
```

```
OLED_Flush();
```

```
HAL_Delay(1000);
```

A.3 Main Loop and Safety Logic

```
uint32_t now = HAL_GetTick();
```

```
float current_A = ReadCurrent_A();
```

```
static uint32_t spike_timer = 0;
```

```
static uint8_t spike_active = 0;
```

```
if (current_A >= CURRENT_LIMIT_A) {
```

```
    if (!spike_active) {
```

```
        spike_active = 1;
```

```
        spike_timer = now;
```

```
    }
```

```
else if ((now - spike_timer) > 400) {
```

```
    if (!g_current_fault) {
```

```
        g_current_fault = 1;
```

```
        g_fault_time_ms = now;
```

```
        g_softstart_active = 0;
```

```
        Motor_Stop(); /* [cite: 17, 30] */
```

```
    }
```

```
}
```

```
} else {
```

```
    spike_active = 0;
```

```
}
```

```
if (g_current_fault) {
```

```

Motor_Stop();

if ((now - g_fault_time_ms) >= FAULT_LOCKOUT_MS) {
    g_current_fault    = 0;
    g_softstart_duty   = 0;
    g_softstart_active = 1;
    g_softstart_tick   = now;
    Motor_SetDirection(g_direction);
}
}

static uint8_t btn_last = 1;
static uint32_t btn_press_time = 0;
uint8_t btn_now = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_11);
if (btn_now == 0 && btn_last == 1) btn_press_time = now;
else if (btn_now == 1 && btn_last == 0) {
    uint32_t duration = now - btn_press_time;
    if (duration > 2000) { g_auto_mode = !g_auto_mode; g_last_flip_ms = now; }
    else if (duration > 50 && !g_auto_mode && !g_current_fault) {
        Motor_SetDirectionWithDeadTime((g_direction == DIR_CW) ? DIR_CCW : DIR_CW);
    }
}

btn_last = btn_now;

if (!g_auto_mode) Update_Encoder_Duty();
if (!g_current_fault) {
    if (g_auto_mode) {
        if ((now - g_last_flip_ms) >= AUTO_REVERSE_MS) {
            if (!spike_active) {
                g_last_flip_ms = now;
                Motor_SetDirectionWithDeadTime((g_direction == DIR_CW) ? DIR_CCW :
DIR_CW);

```

```

        g_target_duty = 93;

    }

}

if (g_softstart_active) Motor_SoftStart(g_target_duty, now);
else Motor_SetDuty(g_target_duty);
}

if ((now - tick_display) >= 200) {
    tick_display = now;

    float safe_current = (current_A < 0.1f) ? 0.0f : current_A;

    display_current_A = (display_current_A * 0.8f) + (safe_current * 0.2f);

    if (display_current_A < 0.05f) display_current_A = 0.0f;

    uint16_t current_active_duty = g_current_fault ? 0 : (g_softstart_active ?
g_softstart_duty : g_target_duty);

    OLED_Update(EstimateRPM(current_active_duty), ((float)current_active_duty /
100.0f) * MOTOR_VOLTAGE, display_current_A, g_current_fault, g_auto_mode, g_direction,
now);
}

HAL_Delay(10);

```

A.4 Physics Math & Custom Functions

```

static float EstimateRPM(uint16_t current_duty) {
    static uint8_t is_spinning = 0;

    const uint16_t START_DUTY = 61; /* 7.3V Breakout threshold */
    const uint16_t STOP_DUTY = 30; /* Coasting threshold */

    if (!is_spinning) {
        if (current_duty >= START_DUTY) is_spinning = 1;
        else return 0.0f;
    } else {
        if (current_duty <= STOP_DUTY) { is_spinning = 0; return 0.0f; }
    }
}

```

```

float lowest_spin_rpm = 2000.0f;

float active_pct = (float)(current_duty - STOP_DUTY) / (100.0f - STOP_DUTY);

float estimated = lowest_spin_rpm + (active_pct * (MOTOR_MAX_RPM -
lowest_spin_rpm));

return (g_direction == DIR_CCW) ? -estimated : estimated;
}

static float ReadCurrent_A(void) {

uint32_t sum = 0;

for (uint8_t n = 0; n < 10; n++) {

    HAL_ADC_Start(&hadc1);

    HAL_ADC_PollForConversion(&hadc1, 10);

    sum += HAL_ADC_GetValue(&hadc1);

    HAL_ADC_Stop(&hadc1);

}

float v = ((sum / 10.0f) / ADC_RESOLUTION) * ADC_VREF;

float i = (v - g_acs_offset) / ACS712_SENS;

return (i < 0.0f) ? -i : i;

}

static void Motor_SetDirection(MotorDir_t dir) {

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, (dir == DIR_CW) ? 1 : 0);

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, (dir == DIR_CCW) ? 1 : 0);

    g_direction = dir;

}

```